

1. Dependencias Necesarias (pom.xml)

Para trabajar de forma unificada con JSON, XML, CSV y las anotaciones de mapeo relacional/objeto, se deben añadir las siguientes dependencias en el archivo de configuración de Maven:

```
<dependencies>
  <!-- Jackson Core y Databind para JSON y Anotaciones -->
  <dependency>
    <groupId>com.fasterxml.jackson.core</groupId>
    <artifactId>jackson-databind</artifactId>
    <version>2.17.2</version>
  </dependency>

  <!-- Soporte para XML -->
  <dependency>
    <groupId>com.fasterxml.jackson.dataformat</groupId>
    <artifactId>jackson-dataformat-xml</artifactId>
    <version>2.17.2</version>
  </dependency>

  <!-- Soporte para CSV -->
  <dependency>
    <groupId>com.fasterxml.jackson.dataformat</groupId>
    <artifactId>jackson-dataformat-csv</artifactId>
    <version>2.17.2</version>
  </dependency>
</dependencies>
```

2. Apuntes Teórico-Prácticos con Ejemplos

Bloque A: Mapeo de Clases Java (POJO) con Anotaciones

Definición de una clase modelo (POJO) utilizando las anotaciones unificadas de Jackson para controlar el nombre de los campos, el orden de las columnas para CSV y la representación de atributos en XML.

```
import com.fasterxml.jackson.annotation.JsonProperty;
import com.fasterxml.jackson.annotation.JsonPropertyOrder;
import com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlProperty;
import com.fasterxml.jackson.dataformat.xml.annotation.JacksonXmlRootElement;

@JacksonXmlRootElement(localName = "alumno") // Elemento raíz en XML
@JsonPropertyOrder({"id", "nombre", "notaMedia"}) // Orden explícito (útil para CSV)
public class Alumno {

    @JsonProperty("id_alumno") // Cambia el nombre de la clave/etiqueta
    @JacksonXmlProperty(isAttribute = true) // En XML se mapeará como atributo
    private int id;

    @JsonProperty("nombre_completo")
```

```

private String nombre;

private double notaMedia;

// Constructor vacío (OBLIGATORIO para frameworks de serialización)
public Alumno() {}

public Alumno(int id, String nombre, double notaMedia) {
    this.id = id;
    this.nombre = nombre;
    this.notaMedia = notaMedia;
}

// Getters y Setters
public int getId() { return id; }
public void setId(int id) { this.id = id; }

public String getNombre() { return nombre; }
public void setNombre(String nombre) { this.nombre = nombre; }

public double getNotaMedia() { return notaMedia; }
public void setNotaMedia(double notaMedia) { this.notaMedia = notaMedia; }

@Override
public String toString() {
    return "Alumno{" + "id=" + id + ", nombre='" + nombre + "'" + ", notaMedia=" +
notaMedia + '}';
}
}

```

Bloque B: Crear un JSON desde un Fichero de Datos Planos (TXT/CSV)

En este escenario leemos un archivo plano de texto delimitado por comas, procesamos sus líneas asignándolas a objetos Java y exportamos el resultado final a un archivo JSON formateado.

Fichero de entrada (alumnos.txt):

```

1,Ana García,8.5
2,Carlos Pérez,6.2
3,Lucía Fernández,9.1

```

Código Java de procesamiento:

```

import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.databind.SerializationFeature;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.IOException;
import java.util.ArrayList;
import java.util.List;

```

```

public class TxtAJson {
    public static void main(String[] args) {
        File archivoTxt = new File("alumnos.txt");
        List<Alumno> listaAlumnos = new ArrayList<>();

        // 1. Lectura del fichero plano
        try (BufferedReader br = new BufferedReader(new FileReader(archivoTxt))) {
            String linea;
            while ((linea = br.readLine()) != null) {
                String[] datos = linea.split(",");
                int id = Integer.parseInt(datos[0].trim());
                String nombre = datos[1].trim();
                double nota = Double.parseDouble(datos[2].trim());

                listaAlumnos.add(new Alumno(id, nombre, nota));
            }
        } catch (IOException e) {
            e.printStackTrace();
        }

        // 2. Escritura en JSON mediante ObjectMapper
        ObjectMapper mapper = new ObjectMapper();
        mapper.enable(SerializationFeature.INDENT_OUTPUT); // Formato "pretty print"

        try {
            File salidaJson = new File("alumnos.json");
            mapper.writeValue(salidaJson, listaAlumnos);
            System.out.println("Fichero JSON generado con éxito.");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Bloque C: Conversión Directa entre JSON y XML

Mediante la estructura en árbol `JsonNode` de Jackson podemos realizar conversiones entre formatos sin necesidad de pasar por clases POJO intermedias.

```

import com.fasterxml.jackson.databind.JsonNode;
import com.fasterxml.jackson.databind.ObjectMapper;
import com.fasterxml.jackson.dataformat.xml.XmlMapper;

import java.io.File;
import java.io.IOException;

public class ConversorJsonXml {

    // Convertir de JSON a XML
    public static void jsonAXml(File jsonFile, File xmlFile) throws IOException {
        ObjectMapper jsonMapper = new ObjectMapper();
        JsonNode tree = jsonMapper.readTree(jsonFile); // Carga el JSON en memoria como
        árbol

        XmlMapper xmlMapper = new XmlMapper();
    }
}

```

```

        xmlMapper.enable(com.fasterxml.jackson.databind.SerializationFeature.INDENT_OUTPUT);
        xmlMapper.writeValue(xmlFile, tree);
    }

    // Convertir de XML a JSON
    public static void xmlToJson(File xmlFile, File jsonFile) throws IOException {
        XmlMapper xmlMapper = new XmlMapper();
        JsonNode tree = xmlMapper.readTree(xmlFile);

        ObjectMapper jsonMapper = new ObjectMapper();

        jsonMapper.enable(com.fasterxml.jackson.databind.SerializationFeature.INDENT_OUTPUT);
        jsonMapper.writeValue(jsonFile, tree);
    }

    public static void main(String[] args) {
        try {
            jsonAXml(new File("alumnos.json"), new File("alumnos.xml"));
            System.out.println("Convertido JSON -> XML");

            xmlToJson(new File("alumnos.xml"), new File("alumnos_desde_xml.json"));
            System.out.println("Convertido XML -> JSON");
        } catch (IOException e) {
            e.printStackTrace();
        }
    }
}

```

Bloque D: Mapeo de Clases Java a CSV

Uso de CsvMapper y CsvSchema para la exportación e importación de colecciones de objetos a archivos CSV con encabezados y delimitadores personalizados.

```

import com.fasterxml.jackson.dataformat.csv.CsvMapper;
import com.fasterxml.jackson.dataformat.csv.CsvSchema;

import java.io.File;
import java.io.IOException;
import java.util.List;

public class MapeoCsv {

    public static void exportarACsv(List<Alumno> alumnos, File destino) throws IOException {
        CsvMapper csvMapper = new CsvMapper();

        // Creamos el esquema a partir de las anotaciones de la clase
        CsvSchema schema = csvMapper.schemaFor(Alumno.class)
            .withHeader() // Incluye la fila de cabecera
            .withColumnSeparator(';'); // Delimitador habitual

        csvMapper.writer(schema).writeValue(destino, alumnos);
    }

    public static List<Alumno> importarDesdeCsv(File origen) throws IOException {

```

```

    CsvMapper csvMapper = new CsvMapper();

    CsvSchema schema = csvMapper.schemaFor(Alumno.class)
        .withHeader()
        .withColumnSeparator(';');

    return csvMapper.readerFor(Alumno.class)
        .with(schema)
        .<Alumno>readValues(origen)
        .readAll();
}
}

```

3. Ejercicios Propuestos para los Alumnos

Ejercicio 1: Migración de Logs de Servidor (TXT a JSON)

Enunciado: Un servidor genera un fichero de log en formato .txt con líneas que siguen la estructura: **TIMESTAMP|NIVEL|MENSAJE** (Ejemplo: 2026-10-05 10:15:00|ERROR|Conexión a BD fallida).

Tarea: Diseña una clase RegistroLog y crea un programa en Java que lea el archivo de log y lo convierta en un archivo logs.json estructurado.

Ejercicio 2: Conversor Bidireccional de Configuración (JSON ↔ XML)

Enunciado: Crea una herramienta en línea de comandos (CLI) que reciba dos argumentos: el archivo de origen y el de destino.

Requisitos:

- Si la extensión de origen es .json y la de destino es .xml, realiza la conversión a XML.
- Si la extensión de origen es .xml y la de destino es .json, realiza la conversión a JSON.
- Controla las excepciones si alguno de los archivos no existe o si la estructura de marcas/JSON no está bien formada.

Ejercicio 3: Mapeo de Objetos Complejos (Java ↔ JSON / XML / CSV)

Enunciado: Diseña un modelo de clases para un módulo de inventario con dos entidades: Producto (id, descripción, precio, stock) y Categoría (id, nombre).

Tareas:

1. Utiliza anotaciones de Jackson para que en el JSON el campo precio se llame 'precio_pvp' y en XML el campo 'id' sea un atributo del nodo.
2. Crea un programa con un menú interactivo por consola que permita:
 - Opción A: Exportar la lista de productos a productos.csv.
 - Opción B: Exportar la lista de productos a productos.json.
 - Opción C: Leer productos.csv e importar los datos de nuevo a objetos Java.